VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

BCSE302L – Database Systems Lab

Course Project

Submitted by:

Student Name	Register Number
Atiksh Chawla	**24BDS0339**
Siddhant Jain	**24BCI0190**
Aujas Shahi	**24BDS0189**

Faculty Coordinator:

Prof. Lydia Jane

August 2026

NexDrive

Review 1: Problem Definition, Requirement Analysis & Database Design

A self-hosted, content-deduplicating cloud storage platform built on MongoDB

1. Problem Statement

Cloud storage services like Google Drive are closed-source and charge per gigabyte. **NexDrive** is a self-hosted cloud storage platform that gives users full control of their data while adding a standout feature — **smart deduplication**: every file is content-addressed by its SHA-256 hash, stored once, and shared across users, so 100 uploads of the same 1 GB file cost 1 GB, not 100 GB.

> *Problem: design and build a multi-user cloud storage web app where users upload, organize, share, trash, and permanently delete files under per-user storage quotas, while globally deduplicating identical content and reporting real-time storage savings.*

1.1 Objectives

- Authenticated multi-user upload, download, and in-app preview (images, video, audio, PDF, text).
- Hierarchical folders — create, rename, move (single/bulk), and delete.
- Content-addressed deduplication: one physical blob per unique content, however many users hold a copy.
- Per-user storage quotas (1 GB default) with live usage and dedup-savings stats.
- Public share links with download counting, plus soft-delete trash (restore / permanent delete / empty).
- Per-user API rate limiting.

1.2 Scope

In scope: authentication, file/folder management, presigned-URL uploads to S3-compatible storage, deduplication, quotas, public sharing, trash, rate limiting, sample dataset.

Out of scope: file versioning, offline sync, real-time collaboration, team/workspace admin, billing, and user-to-user sharing (public links only).

2. Requirement Specification

2.1 Functional

- **Accounts** — register with name/email/password; persistent sessions; optional avatar.
- **Upload & dedup** — client hashes content (SHA-256) before upload (max 10 GB); an existing hash creates a new reference and increments refCount instead of re-uploading.
- **Folders** — create, rename, move, delete (only when empty); listings are newest-first with trashed items hidden.
- **Trash** — soft delete (recursive for folders), restore, permanent delete, empty-trash; a blob leaves object storage only when its last reference is gone.
- **Quotas & stats** — every upload checked against storageUsed + size ≤ storageLimit; dashboard reports files, dedup count, physical vs. logical size, and savings %.
- **Sharing** — toggle a public link with an unguessable share ID; each download increments a counter.
- **Rate limiting** — per-user request cap per 1-second window (default 2 req/s); stale windows expire automatically.

2.2 Non-functional & Assumptions

- Folder/trash listings return in ~200 ms at 10k assets per user; multi-collection writes are atomic (transactions).
- Physical storage scales with unique content, not user count; collections shard horizontally.
- File content is never public — all access is via short-lived presigned URLs; rate-limit data is ephemeral and safe to lose.
- **Assumption:** content lives in S3-compatible object storage (the DB stores only metadata/hashes); MongoDB runs as a replica set; SHA-256 is collision-safe for dedup.

3. NoSQL Database Selection & Justification

Three workload traits point at a **document store**: dedup lookup is a point-read by key (files.findById(hash)); asset metadata is heterogeneous (optional content-type, share ID, trash timestamp); and rate-limit data is high-write and ephemeral — a fit for MongoDB's self-expiring TTL indexes.

3.1 Comparison of NoSQL Options

Criterion	Neo4j	Cassandra	MongoDB (chosen)
Data shape fit	Deep traversal — but folder trees are shallow; forcing files into nodes is bloat.	Wide-column; optional per-asset fields sit awkwardly in a fixed column model.	Native fit — records with optional fields, exactly what assets/files need.
Atomicity	Not its focus.	No multi-collection transactions — breaks ref-count/quota updates.	Multi-document transactions (replica set) cover dedup + quota together.
Ephemeral data	No TTL primitive.	Possible via TTL but heavy for this volume.	TTL indexes expire rate-limit windows for free — no cleanup job.
Auth-stack fit	No first-party integration.	No first-party integration.	user / session / account collections drop in as-is.

Table 1. Only MongoDB satisfies every row — the deciding factor over a graph or wide-column store.

3.2 Design Consequences & Trade-offs

- The 1:1 quota is **embedded** in `user`, updated atomically with `$inc`; the user↔file many-to-many becomes `assets` link documents, so every per-user view is a single-collection query — no joins.
- **Trade-off:** no foreign keys — every `fileHash`/`folderId` invariant is enforced in the API layer inside transactions.
- **Trade-off:** tree traversal (trash/restore of a folder subtree) is application code, not a query primitive — acceptable since trees stay shallow.

4. Data Model Design

4.1 Entities & Relationships

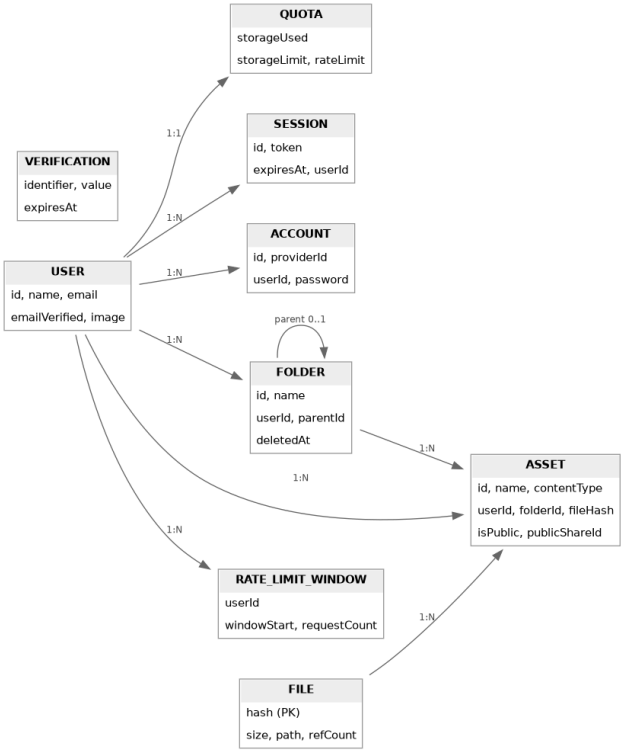


Figure 1. Conceptual model — nine entities, mostly 1:N from USER, plus a self-referencing FOLDER tree.

4.2 ER-to-NoSQL Mapping

Every entity becomes its own collection except QUOTA, which is embedded in `user`. Dashed arrows are reference fields, not joins — resolved in application code.

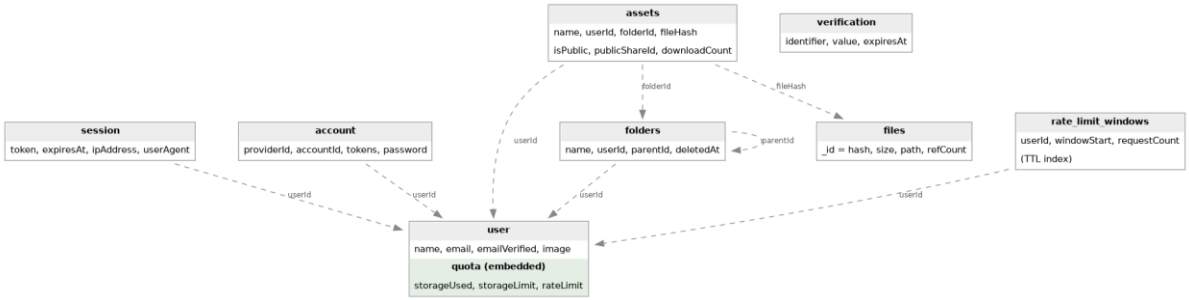


Figure 2. Collection mapping — green = embedded subdocument, dashed arrow = reference field.

- **Embed** the quota only — 1:1 with the user, tiny, read every request, updated atomically.
- **Reference** files from assets — blobs are shared across users; embedding would duplicate size/refCount.
- **Reference** the folder tree via `parentId` (adjacency list) — a move/rename stays a one-field update.
- **Keep sessions, accounts, and rate-limit windows separate** — high write churn that would otherwise bloat the user document.

4.3 Core Collection Schemas (Mongoose)

files — content-addressable blob registry; the SHA-256 hash is the primary key, so a dedup check is one point-read.

```
const FileSchema = new Schema({
  _id: { type: String },              // SHA-256 hash = primary key
  size: { type: Number, required: true },
  path: { type: String, required: true },   // object key in R2
  refCount: { type: Number, default: 1, index: true },
}, { timestamps: true, versionKey: false });
```

folders — adjacency-list tree; a rename or move is a single-field update.

```
const FolderSchema = new Schema({
  name: { type: String, required: true },
  userId: { type: String, required: true },
  parentId: { type: String, default: null },  // null = root
  deletedAt: { type: Date, default: null },   // trash
}, { timestamps: true });
FolderSchema.index({ userId: 1, parentId: 1, deletedAt: 1 }); // folder listing
```

assets — one link document per user-file claim, owning its own metadata (name, sharing, trash state) so per-user views need no joins.

```
const AssetSchema = new Schema({
  name: String, contentType: String,
  userId: { type: String, required: true },
  folderId: { type: String, default: null },
  fileHash: { type: String, required: true },     // -> files._id
  isPublic: Boolean, publicShareId: String, downloadCount: Number,
  deletedAt: { type: Date, default: null },
}, { timestamps: true });
AssetSchema.index({ userId: 1, folderId: 1, deletedAt: 1 });   // folder listing
AssetSchema.index({ publicShareId: 1 }, { unique: true, sparse: true });
```

user embeds the 1:1 **quota** subdocument; **session / account / verification** stay in their own auth collections (written far more often than a user is read); **rate_limit_windows** is a separate, high-churn collection expired automatically by a TTL index:

```
quota: { storageUsed: Number, storageLimit: Number, rateLimit: Number }
RateLimitWindowSchema.index({ createdAt: 1 }, { expireAfterSeconds: 60 }); // TTL
```

4.4 Key Operations

Operation	Implementation
Hash check	files.findById(hash) — one point-read decides dedup vs. new blob.
Upload commit	Txn: files.insertOne + assets.insertOne + user.$inc quota.storageUsed.
Dedup hit	Txn: assets.insertOne + files.$inc refCount + user.$inc quota.storageUsed.
Delete asset	Txn: assets.deleteOne + files.$inc refCount (drop blob at zero) + quota $inc.
Statistics	$match user → $group by fileHash for unique sizes → totals & savings %.
Rate limit	findOne({userId, windowStart}), $inc requestCount; TTL index purges old windows.

4.5 Integrity

- Multi-document transactions wrap every operation touching 2+ collections, so ref-counts and quota deltas never tear.
- Referential integrity is enforced in the API layer: an asset may only reference an existing file and an owned folder.
- Uniqueness via indexes: `user.email`, `session.token`, `assets.publicShareId`, `(userId, windowStart)`.

5. Sample Dataset Design

Alice and Bob each have an account. Alice uploads a resume, a photo, and a video trailer; Bob uploads the **same** trailer — a dedup hit, so his asset points at Alice's blob — plus a photo of his own. Alice organizes files into Personal/Photos, makes her resume public (12 downloads), and has one photo in trash.

5.1 files (_id = SHA-256 hash — the trailer is shared)

```
[ { "_id": "9f2c…a1b3", "size": 10485760,   "refCount": 1 },  // resume.pdf
  { "_id": "d4e5…c2f8", "size": 1048576,    "refCount": 1 },  // photo_a.jpg
  { "_id": "7a1b…e94d", "size": 838860800, "refCount": 2 },  // trailer.mp4 (shared)
  { "_id": "3c6f…b7d2", "size": 5242880,    "refCount": 1 } ] // photo_b.png
```

5.2 assets (one link document per user-file claim)

```
[ { "_id":"a_1", "name":"resume.pdf",   "userId":"u_alice", "folderId":"f_personal",
    "fileHash":"9f2c…a1b3", "isPublic":true,  "publicShareId":"6b8e…6a7b", "downloadCount":12 },
  { "_id":"a_2", "name":"photo_a.jpg",  "userId":"u_alice", "folderId":"f_photos",
    "fileHash":"d4e5…c2f8", "isPublic":false, "deletedAt":"2026-08-19T09:00:00Z" },  // in trash
  { "_id":"a_3", "name":"trailer.mp4",  "userId":"u_alice", "folderId":"f_personal",
    "fileHash":"7a1b…e94d", "isPublic":false },
  { "_id":"a_4", "name":"trailer.mp4",  "userId":"u_bob",   "folderId":"f_work",
    "fileHash":"7a1b…e94d", "isPublic":false },   // dedup hit — same fileHash as a_3
  { "_id":"a_5", "name":"photo_b.png",  "userId":"u_bob",   "folderId":null,
    "fileHash":"3c6f…b7d2", "isPublic":false } ]
```

5.3 user (quota embedded)

```
{ "_id":"u_alice", "name":"Alice Johnson", "email":"alice@example.com",
  "quota": { "storageUsed": 1085276160, "storageLimit": 1073741824, "rateLimit": 2 } }
```

`folders`, `session`, `account`, `verification`, and `rate_limit_windows` follow directly from the schemas in §4.3 — e.g. `f_personal` and its child `f_photos` (both owned by Alice) and `f_work` (Bob, root-level).

5.4 Sanity Checks

- **Dedup:** 4 blobs cover 5 assets — Alice's and Bob's trailers share one 1 GB blob (refCount 2).
- **Quota:** Alice = resume (10 MB) + trailer (1 GB) + trashed photo (1 MB) = 1,085,276,160 B.
- **Trash:** `a_2.deletedAt` is set — the photo appears in Alice's trash and is restorable.
- **Sharing:** `a_1` resolves via `publicShareId`; every fetch increments `downloadCount`.